DAILY DATA DEEP DIVES · THE LATEST NEWS, FULLY EXPLAINED

Storage Wars

pgBackRest stops restoring corrupt pages, and how Pinterest knows when a streaming hour is finished

Issue #16 | Tue, 20 Oct 2026 | ⏱️ ~34 min read | Level 3/5

🗄️ Today's two stories

1. **pgBackRest 2.60.0, released on 18 October, changes what happens when a Postgres backup contains pages with bad checksums.** Until now it warned and carried on. Now `restore` stops with an error and leaves the cluster unable to start, and `expire` refuses to delete older backups if that would leave a corrupt one as your oldest. We explain how Postgres page checksums work, why a backup tool can catch corruption your database hasn't noticed yet, how the new `checksum-page-error` option behaves in each command, and what to check before you upgrade.
2. **Pinterest explained how its streaming change-data pipelines tell downstream batch jobs that an hour of data is complete.** No batch job "finishes" any more, so the Flink-to-Iceberg sink records event-time statistics, a few hundred bytes per commit, and publishes a watermark that never moves backwards. We cover why "is this partition done?" is hard in streaming, how the watermark is computed and stored, and the trade-off between completeness and freshness.
3. **On the radar:** 6 more things that happened, listed at the end. The first one is urgent if you run Redis.

STORY 1 pgBackRest learns to say no

POSTGRESQL · PGBACKREST 2.60.0 · RELEASED 18 OCT 2026 · ~15 MIN READ

Picture the worst morning of your on-call year. The primary is gone, you start a restore from last night's backup, and an hour later Postgres is up and taking traffic. Three weeks after that, a query against an old partition fails with "invalid page". The corruption was in the backup all along. pgBackRest saw it on the night of the backup, wrote a warning into a log file nobody reads, and restored it for you anyway. Version 2.60.0 changes that default, and it changes it on purpose.

WHAT HAPPENED

On **18 October 2026**, the pgBackRest community released **pgBackRest 2.60.0**, the latest version of the open-source PostgreSQL backup and restore tool.[1] The headline of the release is a new option, `checksum-page-error`, "to error on invalid page checksums", contributed by Andrew Pogrebnoi.[1] The announcement's "Important Notes" spell out what it does by default:

> The restore command will now error and stop if any restored file contains an invalid page checksum, which leaves the cluster incomplete and unable to start, so the restore must be rerun. The expire command will now refuse to expire if the oldest retained backup would contain

checksum errors, which also prevents archive from being expired in that repository. Set checksum-page-error=n to warn instead of erroring.[1]

The project's home page now puts it in one line: by default, "restore errors when the backup has invalid page checksums", and expiry won't leave a backup with invalid page checksums as the oldest one you keep.[2]

The release has other changes too:[1]

- Support for **GCS Workload Identity Federation**, so backups to Google Cloud Storage can authenticate without long-lived keys.
- **Prefetching** of multi-file and range reads from object stores, and fewer keep-alive requests to remote processes.
- Retries for Azure batch sub-requests that are missing from the response.
- Support for OpenSSL 4 and for building on illumos.
- Building from a git checkout now needs PyYAML instead of libyaml. The release tarball needs neither.
- PostgreSQL 19 RC1 is supported by 2.59.2 and later, including 2.60.0.

The checksum change is the one that can alter what happens during an incident, so it's the one this story is about.

THE BACKGROUND YOU NEED

What a page checksum is

PostgreSQL stores tables and indexes in fixed-size **pages** *≈ the 8 KB blocks that Postgres reads and writes; every table and index file is a sequence of them*. With **data checksums** *≈ a small value stored in each page's header, computed from the page's contents when it's written and checked every time it's read* turned on, "each data page includes a checksum that is updated when the page is written and verified each time the page is read".[7] If a disk, a controller, a filesystem bug or a stray `dd` changes even one byte of a page, the next read of that page notices.

Checksums protect data pages only: "internal data structures and temporary files are not" covered. They're switched on or off for the whole cluster, never per database or table, and you can check the current state with `SHOW data_checksums`.[7]

For most of Postgres's history, checksums were off unless you asked for them. The PostgreSQL 17 docs still open with "By default, data pages are not protected by checksums".[8] **PostgreSQL 18**, released on **25 September 2025**, flipped that: `initdb` now enables checksums by default.[9] So every cluster created fresh on 18 has them. A cluster that was created on 17 or earlier and upgraded with `pg_upgrade` usually doesn't, because `pg_upgrade` requires the old and new clusters to have the same setting.[9,10] To add checksums to an existing cluster, you run **pg_checksums** *≈ a Postgres command-line tool that can enable, disable or verify checksums, but only while the cluster is shut down* offline, or build a replica with checksums and fail over to it.[7,10]

PostgreSQL 19 was going to let you turn checksums on while the database is running. That feature was reverted from the 19 branch on **16 September 2026** after many post-commit fixes during beta. The revert

touched 82 files and deleted 6,673 lines.[12] (Issue #13 covered the wider wave of PG 19 reverts.) For now, offline is still the only way.

Why corruption hides

A checksum is only checked when a page is **read**. Postgres doesn't scan your whole database on a schedule. So a page in a rarely touched partition, an old index, or a table you only query at quarter-end can sit corrupt on disk for months. The first thing to notice it might be:

- A user query, which fails, or a **VACUUM** ≈ *Postgres's background maintenance that cleans up dead rows; it reads pages as it goes and so can trip over corruption* run.
- Your backup tool, which reads every file it copies.

Postgres counts the failures it sees. Since a 2019 change by Magnus Hagander, `pg_stat_database` has a `checksum_failures` column per database. It counts failures found by normal reads and failures found during base backups.[13] If you've never looked at that column, now is a good time:

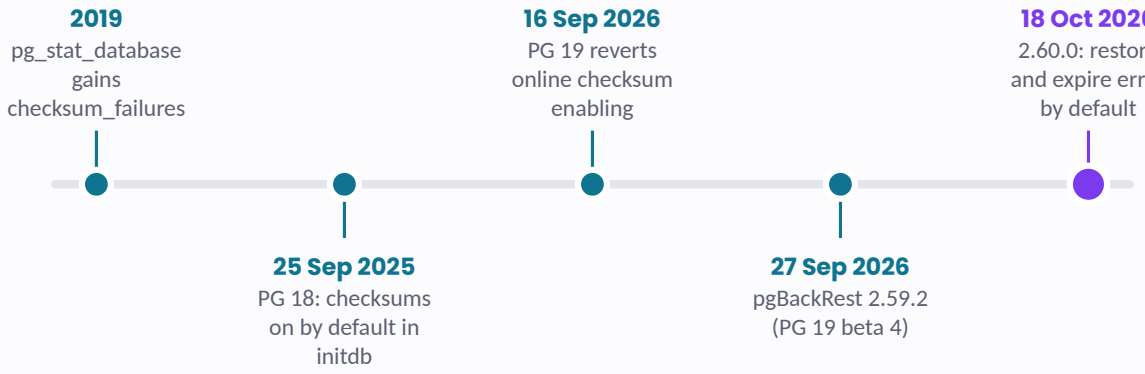
```
SELECT datname, checksum_failures, checksum_last_failure
FROM pg_stat_database
WHERE checksum_failures > 0;
```

What pgBackRest already did

Checking checksums during backup isn't new for pgBackRest. Version 1.12 added a `--checksum-page` option to "validate page checksums in data files when checksums are enabled". It's switched on automatically when the cluster has checksums.[4,5] Full backups check every page. Differential and incremental backups check pages in the files that changed. Until now, though, a failure didn't stop anything. "Validation failures don't stop the backup", and the failing pages were reported as warnings in the console and the log file.[2]

Postgres's own `pg_basebackup` takes a similar line. When a base backup finds checksum failures, it logs warnings, keeps going, and reports an error at the end with a non-zero exit code, but it keeps the backup directory.[14] In both tools the backup carries on: you get your copy, plus a note saying part of it is bad. 2.60.0 is about what pgBackRest does with that note later.

Checksums and backups: how we got here



Sources: PostgreSQL commit 6b9e875 (2019), PostgreSQL 18 release announcement, REL_19_STABLE revert commit c05d5ce, pgBackRest GitHub releases, pgBackRest 2.60.0 announcement.

HOW IT WORKS

One option, three commands, three behaviours

The pull request that added the option, #2833, applies `checksum-page-error` to three commands. Each one has its own default and its own moment of failure:[3]

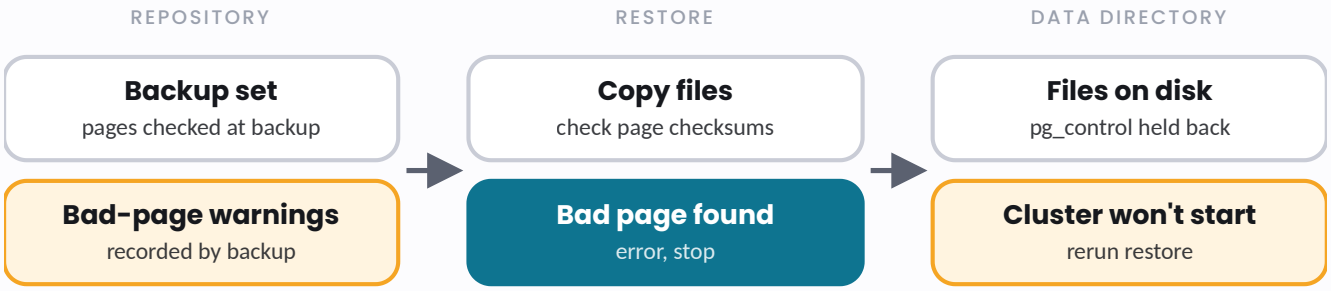
Command	Default	What "error" means
`backup`	**off**	Stop at the first invalid page checksum (or misaligned file). Off: one warning per file, plus a total at the end.
`restore`	**on**	Stop at the first file with invalid checksums, **before `pg_control` is renamed**, so the cluster can't start. Off: warnings and a total at the end.
`expire`	**on**	If the backup that would become the oldest one kept has invalid checksums, stop before removing anything. Off: warn instead.

The `backup` default hasn't changed. Your nightly job still finishes and still writes a backup when it finds bad pages, just as before. The new strictness happens when you **use** the backup (`restore`) and when you **throw away** the good ones (`expire`).

Why restore fails before `pg_control`

Every Postgres data directory contains `global/pg_control`, a small file holding the cluster's identity and state, which Postgres reads first at startup. pgBackRest's restore stops before that file is put in its final place. So when the restore errors, the directory is deliberately left in a state where Postgres won't start, and you can't bring up a half-checked cluster by accident.[3,1] The maintainers in the review agreed that this was the point: a bad backup shouldn't be restored into a cluster that can start.[3]

Restore in 2.60.0: corrupt pages now stop the line



With checksum-page-error=n the restore completes, prints warnings and a total, and the cluster can start with the bad pages in place.

What do you do when it happens? You have three choices. You can restore a **different** backup set, an older or newer one, whose pages are clean. You can rerun the restore with `--checksum-page-error=n` if you've decided you'd rather have the data with known bad pages than no data. Or you can investigate first: which files, which tables, and whether those pages are also bad on the primary or a replica.

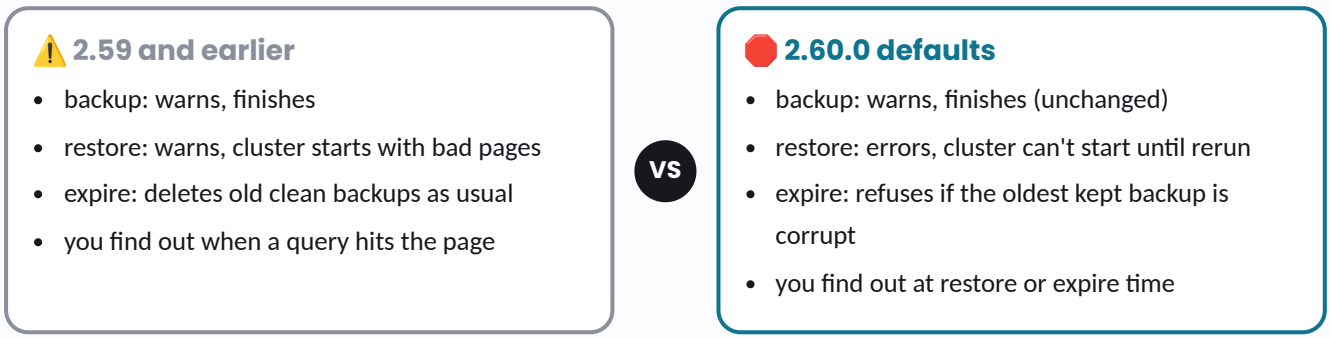
Why expire refuses

Retention ≈ *the policy that decides how many backups to keep; pgBackRest's expire command deletes backups, and the WAL they depend on, once they fall outside it* is where a corrupt backup does the most quiet damage. Suppose you keep two full backups. Last night's full backup contains a few bad pages. Tonight a new full backup runs, and `expire` deletes the oldest one, which was clean. Now your two retained backups are tonight's and last night's. If the corruption is on the primary's disk itself, both are probably bad, and the last good copy has just been deleted.

2.60.0's `expire` checks the backup that would become the **oldest** retained one. If it has invalid checksums, `expire` stops before removing anything.[3] Because the WAL archive is trimmed along with the backups, archive expiry stops too.[1] That gives you a growing repository, a loud error, and your last good backup still in place. A backup that warns you is better than one that cleans up the evidence.

There's a cost: a repository that stops expiring keeps growing. If nobody reacts to the error, your storage bill and disk usage climb until someone does. That's the trade-off the maintainers chose.

A corrupt backup, before and after 2.60.0



Based on the pgBackRest 2.60.0 announcement and pull request #2833.

What counts as an invalid page, and what doesn't

A backup reads files while Postgres is writing to them, so a backup tool has to avoid false alarms. A page might be half-written when the backup reads it. This is a **torn page** *≈ a page caught in the middle of being written, so part of it is old and part new; its checksum won't match until the write finishes*, and it gets fixed when WAL is replayed during recovery. Postgres's `pg_basebackup` handles this by checking only pages whose **LSN** *≈ Log Sequence Number: a position in the write-ahead log; each page records the LSN of the last change written to it* is older than the backup's start, and by re-reading a failed block once before counting it.[14]

pgBackRest's own notes on the subject say files may grow or shrink during a backup, which is normal. It reads each file only up to the size recorded when it built the backup manifest, and skips blocks added after that, because "those new blocks may be torn".[6] It also follows Postgres in treating all-zero pages as pages with no checksum.[6] The checks are built to flag real damage, not ordinary churn. If a backup reports bad pages, take it seriously.

Keeping the old behaviour, if you must

pgbackrest.conf: 2.60.0 default

```
[global]
repo1-path=/var/lib/pgbackrest
repo1-retention-full=2

# checksum-page-error not set:
# restore -> error on bad pages
# expire  -> refuse if oldest
#            kept backup is bad
```

pgbackrest.conf: warn only

```
[global]
repo1-path=/var/lib/pgbackrest
repo1-retention-full=2

# pre-2.60 behaviour
checksum-page-error=n
```

Illustrative config; checksum-page-error=n is the documented switch from the 2.60.0 announcement. Prefer setting it per command on the command line during an incident rather than globally.

WHAT IT MEANS FOR YOU

If your clusters don't have checksums, nothing changes. No checksums means no page checks during backup, so there's nothing to error on. Run `SHOW data_checksums` on each cluster. Anything created on PostgreSQL 18 or later will probably say `on`, and anything `pg_upgrade`d from an older cluster may not.[9]

If they do, look at your recent backup logs before upgrading. Search for checksum warnings from your existing backups. If an existing backup in your repository already has bad pages, it'll start mattering the first time `expire` runs under 2.60.0 or you try to restore it. The PR review flagged that the restore check applies to backups made by **older** pgBackRest versions too.[3] That's what you'd want, but it can still catch you off guard on your first restore after upgrading.

Add an alert on expire failures. Your backup monitoring probably checks that the nightly backup succeeded. Under 2.60.0, the new failure mode is the `expire` step erroring while backups keep succeeding, with the repository quietly growing. Make sure an `expire` error pages someone, and check repository size growth too.

Rehearse the restore. A restore that fails by design during an incident is safe, but it's stressful if nobody knew it could happen. Put the three options above (another backup set, `--checksum-page-error=n`, or investigate first) in your runbook, along with the `pg_stat_database` query. If your restore tests run on a schedule, 2.60.0 has just made them more meaningful: a test restore that used to "pass" with warnings will now fail.

Wrappers and operators need checking. Many teams run pgBackRest through Kubernetes operators or in-house wrappers. Check how your tooling surfaces a non-zero exit from `restore` or `expire`, and whether it passes extra options through. Whoever packages your pgBackRest decides when you get 2.60.0. The announcement points to the PGDG yum and apt repositories.[1]

💥 Watch out for

- **The option means different things per command.** It defaults off for `backup` and on for `restore` and `expire`, and each fails at a different moment. A reviewer called this out as confusing semantics for one option name.[3]
- **Delta and force restores may not count everything.** The review noted that corrupt files preserved in place during `--delta` or `--force` restores aren't counted, so those restores can finish without a warning even though bad pages are on disk.[3] Check the final 2.60.0 docs for whether this was addressed before release.
- `--no-checksum-page` **plus** `checksum-page-error` **does nothing.** If page checks are off for backup, there's nothing to error on. The reviewer suggested making one depend on the other.[3]
- **A stuck expire means a growing repository.** That's intended, but it costs disk or object storage until someone acts.
- **Checksums only cover data pages.** Temporary files and internal data structures aren't covered by data checksums at all.[7]

WHAT HAPPENS NEXT

The review on PR #2833 raised some open points: how the expire check finds the oldest retained backup, whether the error message should tell you how to get around it, and how the manifest records a file's error state when a backup aborts.[3] Watch the next point releases for refinements. On the Postgres side, PostgreSQL 19's GA is planned for **19 November**,[11] and pgBackRest 2.59.2 and later already support the release candidate.[1] Online checksum enabling will have to wait for PostgreSQL 20 at the earliest, so the way to add checksums to an existing cluster without long downtime is still a replica with checksums and a failover.[10]

📖 Words from this story

Pages: the 8 KB blocks that Postgres reads and writes; every table and index file is a sequence of them.

Data checksums: a small value stored in each page's header, computed when the page is written and checked every time it's read.

pg_checksums: a Postgres tool that can enable, disable or verify checksums, but only while the cluster is shut down.

VACUUM: Postgres's background maintenance that cleans up dead rows, reading pages as it goes.

Retention: the policy deciding how many backups to keep; pgBackRest's `expire` deletes backups (and the WAL they depend on) outside it.

Torn page: a page caught in the middle of being written, part old and part new, whose checksum won't match until the write completes.

LSN: Log Sequence Number, a position in the write-ahead log; each page records the LSN of its last change.

STORY 2

When is an hour of streaming data done?

KAFKA · FLINK · ICEBERG · PINTEREST ENGINEERING, 25 SEP 2026 (INFOQ, 19 OCT) · ~14 MIN READ

In the batch world, nobody had to ask whether yesterday's data was ready. A nightly job ran, finished, and its finishing was the answer. Then Pinterest replaced those jobs with streaming pipelines that write a little data every few minutes and never finish. That left a question with no obvious owner: when can the job that builds tomorrow's dashboards safely read hour 14:00? Pinterest's answer is a watermark that lives inside the table itself.

WHAT HAPPENED

On **25 September 2026**, Pinterest Engineering published "Partition Finalization in Pinterest's Next-Generation DB Ingestion Framework", by Qianrui Zhang, Kanchi Masalia, Liang Mou and Yi Pan.[15] InfoQ covered it on **19 October**.[16] It's the third post in a series about the platform that copies Pinterest's online databases into its analytics lake. The first post described the platform and the second automated schema evolution.[15,18]

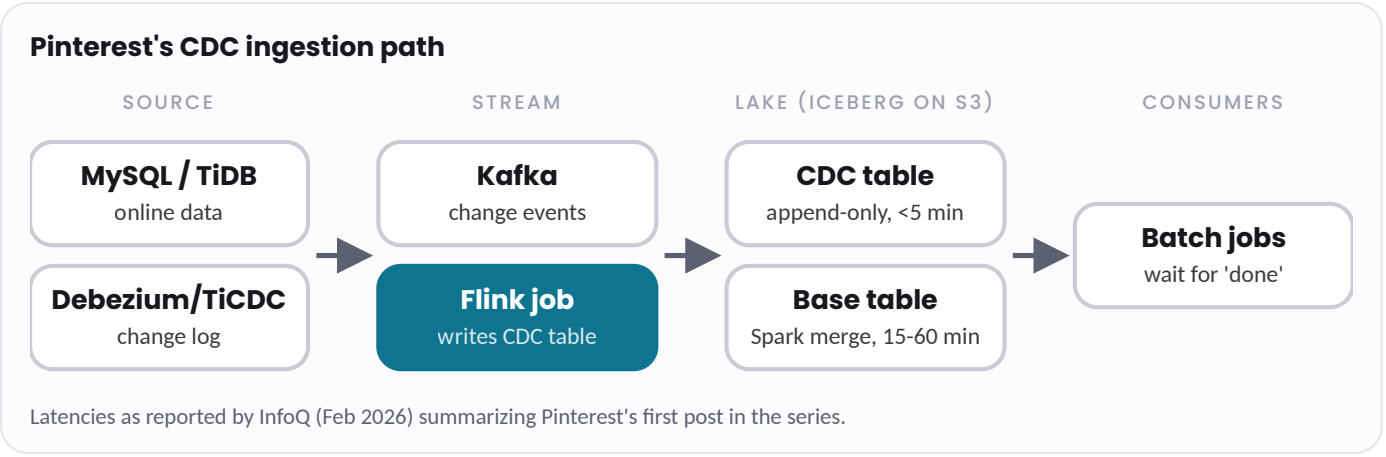
The post describes **event-time-based partition finalization**. The streaming job that writes each table now publishes a signal saying "everything up to hour H has arrived", and downstream jobs wait for that signal before they read. In Pinterest's words, "Finalization is no longer a byproduct of a job finishing."[16] The signal is stored as table metadata in **Apache Iceberg** *≈ an open table format that turns a pile of files in object storage into a table with snapshots, schema and transactions*, so there's no separate coordination service to run.[15]

THE BACKGROUND YOU NEED

Pinterest's ingestion pipeline

Pinterest's online data lives in MySQL, TiDB and an internal key-value store. The old way to get it into the lake was a set of full-table batch jobs. Latency often exceeded 24 hours, even though fewer than 5% of rows in many tables changed on a given day.[17]

The new platform uses **change data capture** *≈ CDC: reading a database's change log so every insert, update and delete becomes an event other systems can consume*. Debezium or TiCDC captures changes and writes them to **Kafka** *≈ a distributed log that stores streams of events durably and lets many consumers read them*. A **Flink** *≈ a stream-processing engine that runs long-lived jobs over unbounded event streams, with periodic consistent checkpoints* job writes them into an append-only **CDC table** in Iceberg on S3, typically within five minutes. A Spark job then periodically merges the latest changes into a **base table**, a full copy of the source table, every 15 minutes to an hour.[17] It runs across "thousands of pipelines" at "petabyte-scale".[16]



Event time versus processing time

Every change event has two clocks attached. **Event time** *≈ when the change actually happened in the source database* is when the row changed. **Processing time** *≈ the clock on the machine handling the event right now* is when the pipeline handles it.[19] They drift apart all the time: a source replica lags, a Kafka partition backs up, a Flink job restarts and replays. Analysts partition by event time, because "orders placed between 14:00 and 15:00" is a question about event time. So a change that happened at 14:59 can arrive at 15:20, after its hour has ended. Pinterest names exactly this problem: data can keep arriving after a partition's wall-clock window has closed.[16]

Stream processors have a standard tool for this, the **watermark** *≈ a marker saying event time has reached t, meaning no more events with timestamps at or before t are expected*. In Flink's definition, a `Watermark(t)` "declares that event time has reached time t in that stream", so "there should be no more elements from the stream with a timestamp t' <= t". Events that turn up after that are **late**.[19] When an operator reads several parallel inputs, "its current event time is the minimum of its input streams' event times". The slowest input sets the pace.[19]

Flink's watermarks live inside a running job, though. Pinterest's problem is between jobs: the Flink job that writes the table and an Airflow-style batch job, hours later, that wants to read it. The batch job can't see inside Flink. It needs the watermark written somewhere durable that it can read.

How the batch world did it

Hadoop-era pipelines solved this with a marker file. When a Spark or MapReduce job commits its output, Hadoop's `FileOutputCommitter` writes an empty `_SUCCESS` file into the output directory. It's controlled by `mapreduce.fileoutputcommitter.marksuccessfuljobs`, which is on by default, and downstream jobs wait for it to appear.[20] Pinterest's old system worked the same way in spirit: a partition was final once the single job producing it finished.[15] A streaming job that commits every few minutes forever never reaches that moment.

Two ways to know a partition is complete

📦 Batch: job completion

- one job writes the whole partition
- it finishes, writes _SUCCESS
- complete by construction
- but latency is the job's schedule (24 h+)

VS

🌊 Streaming: event-time watermark

- many small commits, forever
- completeness must be inferred from event times
- late data is possible, so it's a policy
- latency close to the stream's

Based on Pinterest's post and the Spark _SUCCESS marker's documented behaviour.

Iceberg snapshots and table properties

Two parts of Iceberg's metadata make Pinterest's design work. Every commit to an Iceberg table creates a **snapshot**, "a snapshot of the data in a table at a point in time". Each snapshot carries a **snapshot summary** ≈ *a string-to-string map stored with every Iceberg snapshot, describing the operation that produced it*, a "string map of summary data for the operation that produced this snapshot".[21] The table as a whole also has **table properties** ≈ *a string-to-string map of table-level settings stored in Iceberg's metadata file, readable by any engine*. Summaries are per commit and append-only. Properties are table-wide and can be updated.

When Flink writes to Iceberg, it commits at **checkpoints** ≈ *Flink's periodic, consistent snapshots of a job's state, used to recover after failure*. Writer tasks flush their data files when a checkpoint is triggered, and once the checkpoint completes, a single committer adds all those files to the table as one new snapshot.[22] So each Iceberg snapshot written by Flink corresponds to one checkpoint window, and that's where Pinterest attaches its statistics.

HOW IT WORKS

Pinterest splits the design into **facts** (what happened in each commit) and an **opinion** (whether a partition is done).[15] Facts are recorded once, cheaply. The opinion is computed from them by a policy you can change.

Step 1: extract event time from every record

Each pipeline is configured with an event-time column. A pluggable extractor reads that column from each CDC record and turns it into a timestamp.[15] The table might call it `updated_at` or `event_ts`, so configuration names the column.

Step 2: collect facts per checkpoint

Inside each Flink writer task, lightweight **accumulators** ≈ *small aggregators that run inside each parallel task and are merged into one result at the end* track the event times seen during the checkpoint window: the **minimum**, the **maximum**, and percentiles such as p95 and p99.[15] The percentiles come from a **t-digest** ≈ *a compact sketch of a distribution, invented by Ted Dunning, that estimates percentiles accurately (especially at the tails) and can be merged*.[15,23]

The t-digest choice matters. Each Flink job runs many parallel writer tasks, and each sees only its share of records. Minimums and maximums merge trivially: take the min of the mins and the max of the maxes. Percentiles don't. You can't average two p99s. A t-digest can be merged: in Dunning and Ertl's paper, separately computed digests combine "with no loss in accuracy", and accuracy is best near the tails, which is where late data lives.[23] Pinterest says each sketch stays at "a few hundred bytes".[15]

Step 3: store the facts in the snapshot summary

At the committer, the accumulators from all writer tasks are merged in an order-independent way. The result goes into the Iceberg snapshot summary under a namespaced key.[15] Every snapshot now carries its own event-time profile: "this commit contained events from 13:58 to 14:31, p95 at 14:24". It's permanent and costs only a few hundred bytes.

Step 4: form the opinion after each commit

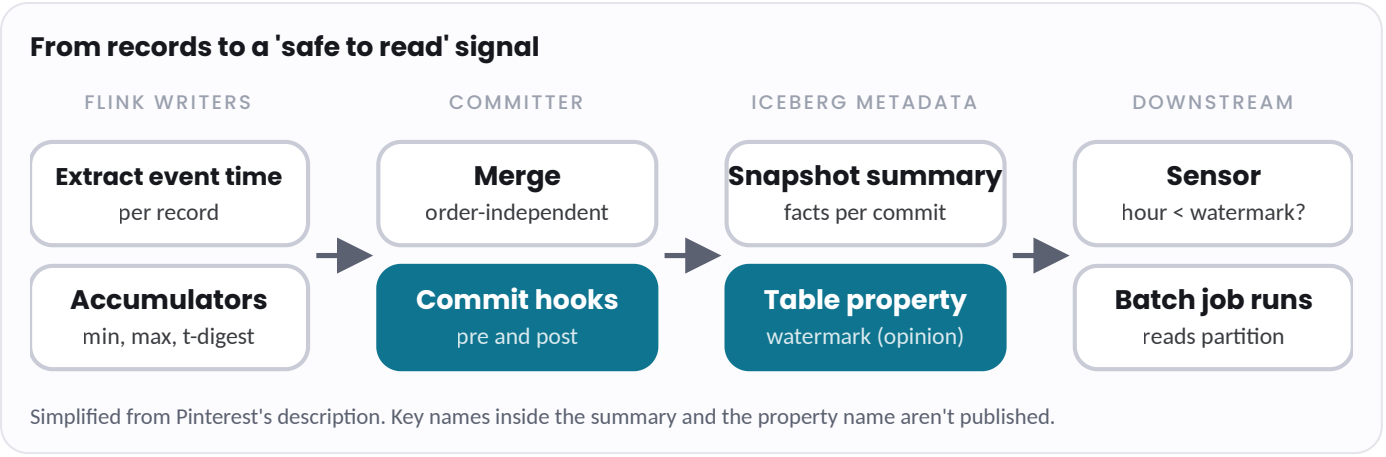
Pinterest wraps each Iceberg commit in **commit processors**, with a pre-commit hook and a post-commit hook. The pre-commit hook can add snapshot properties or abort the commit. The post-commit hook runs the watermark algorithm and updates a table property if the watermark has moved forward.[15]

The default policy is conservative: take the **minimum** of the per-commit minimum event times over the last X commits, and round it down to the hour.[15] Here's a worked example. The numbers and the choice of X=3 are made up for illustration:

Commit (processing time)	Min event time in commit	Max event time
15:05	14:52	15:04
15:10	14:58	15:09
15:15	15:01	15:14

The minimum over the last three commits is 14:52, which rounds down to **14:00**. The watermark says event time has reached 14:00, so every hour before it, up to and including the 13:00–14:00 partition, is finished. The 14:00 hour isn't, because the 15:05 commit still contained a 14:52 event. A few commits later, once every recent commit's minimum is past 15:00, the watermark moves to 15:00 and the 14:00 hour is final.

The watermark is **monotonically non-decreasing**. Once an hour is declared final, it stays final. If late data would pull the watermark backwards, the system doesn't regress it. Instead, that late data can raise an alert.[15] Downstream jobs never see an hour go from "done" back to "not done".



Step 5: store the opinion where readers already look

The watermark is stored as a single Iceberg **table property**, "which downstream jobs read as ordinary metadata".[15] A downstream **sensor** *≈ an orchestrator task (as in Airflow) that waits and re-checks until some condition is true before letting the next task run* loads the table metadata, reads one property and compares it to the hour it wants. No API, no database of partition states, no extra service to keep available.

Step 6: carry it into the base table

Most consumers don't read the CDC table. They read the merged base table. So the Spark upsert job copies the watermark from the CDC table to the base table **in the same commit** that merges the data.[15] That detail matters. The base table's watermark only ever describes data that is actually in the base table, because the two are updated in one atomic Iceberg commit.

Completeness versus freshness

The minimum-based default waits for the slowest record. For tables where freshness matters more, Pinterest lets a pipeline use a percentile instead, such as the p95 or p99 event time, "accepting that a small tail of late records may arrive after their hour was marked safe to read".[15] That's the classic streaming trade-off, made explicit and configurable per table.

The approach also fits inside the existing pipeline. The operator topology is unchanged (writer and committer), and with no extension points configured the sink runs the standard operators with no overhead.[15] Nothing in it is specific to database ingestion. Any Flink-to-Iceberg pipeline can use it by supplying an event-time extractor, and Pinterest plans to apply it to its stream ingestion framework next.[15]

WHAT IT MEANS FOR YOU

If you run Flink into Iceberg (or Delta, or Hudi), you probably have this problem already. Many teams paper over it with a fixed delay: "the 14:00 partition is ready at 16:00". That's fine until a backlog of more than two hours, when it silently produces incomplete reports. It also wastes freshness on the 99% of days when data arrives in minutes. A watermark derived from the data itself fails safe: if the pipeline stalls, the watermark stalls, and downstream jobs wait instead of reading half an hour.

Keep facts separate from opinions. The most reusable idea here isn't the t-digest, it's storing raw per-commit facts permanently and computing the "done" decision from them with a policy you can change. If you later decide a table needs p99 instead of minimum, or a different window X, you recompute from facts you already have. You don't have to reprocess any data.

You can approximate it today without custom sink code. Iceberg exposes snapshot summaries and table properties to every engine, so the storage side is standard. Two open proposals in the Iceberg project would make the Flink side easier: a `WriteObserver` interface that lets a plugin see each record and add per-checkpoint metadata to the snapshot summary, and a `PostCommitHook` that runs after each commit with the snapshot ID and summary. One of its stated examples is "writing watermark metadata to table properties".[24,25] Both describe today's alternative as subclassing internal sink classes, which is fragile across Iceberg upgrades.[24,25]

Decide what late data means for your consumers. Under the minimum policy, a record that arrives after its hour was finalized is a genuine anomaly, and Pinterest alerts on it.[15] Under a percentile policy, it's expected. Either way, a downstream job that already ran won't see that record unless something reprocesses the hour. Know which tables are which.

> 💥 **Watch out for**
>
> - **The watermark is an inference, not a guarantee.** "No recent commit contained anything older than 14:00" doesn't prove nothing older will come. A source replica that's hours behind, or a backfill, can break the assumption. The non-regressing rule keeps consumers consistent, but it can't make late data appear in partitions that were already read.
> - **Idle pipelines stall the signal.** If a table gets no changes for a while, there are no new commits to advance the watermark. Pinterest's post doesn't describe idle handling, so if you build this, decide how quiet tables should advance.
> - **Event time must be meaningful.** The extractor reads a column you configure. An `updated_at` that the application sets inconsistently, or that's in the wrong time zone, produces a confident but wrong watermark.
> - **It's Pinterest's internal code.** The two Iceberg extension proposals are proposals, not released APIs. Pinterest hasn't published key names, the window size X, or measured results for the finalization feature.

WHAT HAPPENS NEXT

Pinterest says the next post in the series will cover incremental processing on CDC pipelines, meaning downstream jobs that process only what changed rather than whole partitions.[15] That's the natural next step: once each commit carries facts about its contents, consumers can decide not only whether an hour is done but what changed since they last read it. Watch the Iceberg project's Flink sink proposals too. If something like `WriteObserver` and `PostCommitHook` lands, this pattern becomes configuration instead of a fork.[24,25]

📖 Words from this story

Apache Iceberg: an open table format that turns files in object storage into a table with snapshots, schema and transactions.

Change data capture (CDC): reading a database's change log so every insert, update and delete becomes an event.

Kafka: a distributed log that stores streams of events durably for many consumers.

Flink: a stream-processing engine that runs long-lived jobs over unbounded streams, with periodic checkpoints.

Event time: when a change actually happened in the source.

Processing time: the clock on the machine handling the event now.

Watermark: a marker saying event time has reached t, so no more events at or before t are expected.

Snapshot summary: a string map stored with each Iceberg snapshot, describing the operation that produced it.

Table properties: a table-level string map in Iceberg's metadata, readable by any engine.

Checkpoints: Flink's periodic consistent snapshots of job state, used for recovery; Iceberg commits happen at them.

Accumulators: small aggregators inside each parallel task, merged into one result.

t-digest: a compact, mergeable sketch of a distribution that estimates percentiles accurately, especially at the tails.

Sensor: an orchestrator task that waits until a condition is true before the next task runs.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **Redis CVE-2026-25243 is now being exploited:** on 19 October, Canada's Cyber Centre updated its advisory to say this `RESTORE` memory bug, patched on 5 May (Redis 6.2.22 / 7.2.14 / 7.4.9 / 8.2.6 / 8.4.3 / 8.6.3 and later; Valkey 8.1.7 / 9.0.4), "is being exploited in the wild", so patch now and, as Redis and Red Hat recommend, deny `RESTORE` (part of `@dangerous`) to users who don't need it.[26,27,28]
- **Kafka 4.4.0 is quietly out:** its artifacts reached Maven Central on 14 October and Spring Boot moved to it on 15 October, but kafka.apache.org's downloads page and blog still list 4.2.2 (29 Sep) as the newest release.[29,30,31]
- **Snowflake Online Feature Store is GA (20 Oct):** low-latency serving of the latest ML feature values from batch, stream and real-time feature views, in AWS and Azure regions but not Google Cloud.[32]
- **DuckDB 2.0.0 is still scheduled for 21 October (tentative):** community extensions are moving their builds to v2.0.0, with a warning that extensions that work on v1.5 may not build against 2.0.[33,34]
- **Snowflake Openflow endpoints are GA (16 Oct):** stable HTTPS URLs that route external HTTP or OTLP traffic to Listen processors in an Openflow runtime, authenticated with a service user's PAT.[35]
- **Databricks `READ METADATA` (16 Oct):** the privilege now supports all compliance security profile standards and is on by default for compliance-profile workspaces.[36]

1. **pgBackRest 2.60.0 Released**, pgBackRest Community, postgresql.org, 18 Oct 2026.
2. **pgBackRest: Reliable PostgreSQL Backup & Restore**, pgbackrest.org (feature list, viewed 20 Oct 2026).
3. **Add checksum-page-error option to error on invalid page checksums (#2833)**, pgbackrest/pgbackrest, GitHub (description and review).
4. **v1.12: Page Checksums, Configuration, and Bug Fixes**, pgbackrest/pgbackrest, GitHub.
5. **Backup Command (backup): checksum-page**, pgBackRest command reference mirror, Pigsty docs.
6. **Improve page checksum validation (#952)**, pgbackrest/pgbackrest, GitHub.
7. **Data Checksums**, PostgreSQL 18 documentation.

8. **Data Checksums**, PostgreSQL 17 documentation.
9. **PostgreSQL 18 Released!**, PostgreSQL Global Development Group, 25 Sep 2025.
10. **Postgres 18 New Default for Data Checksums and How to Deal with Upgrades**, Greg Sabino Mullane, Crunchy Data, 11 Dec 2025.
11. **PostgreSQL 19 RC 1 Released!**, PostgreSQL Global Development Group, 15 Oct 2026.
12. **pgsql: Revert online data checksum transitions**, Daniel Gustafsson, pgsql-committers, 16 Sep 2026.
13. **Track block level checksum failures in pg_stat_database**, PostgreSQL commit by Magnus Hagander (2019 development cycle), mirror.
14. **Validate page level checksums in base backups**, PostgreSQL commit by Michael Banck, committed by Magnus Hagander, mirror.
15. **Partition Finalization in Pinterest's Next-Generation DB Ingestion Framework**, Qianrui Zhang, Kanchi Masalia, Liang Mou and Yi Pan, Pinterest Engineering, 25 Sep 2026.
16. **Pinterest Uses Event Time Watermarks to Finalize Petabyte Scale Data Partitions**, Leela Kumili, InfoQ, 19 Oct 2026.
17. **Pinterest's CDC-based database ingestion framework**, Leela Kumili, InfoQ, 26 Feb 2026.
18. **Automated Schema Evolution in Pinterest's Next-Generation DB Ingestion Framework**, Yisheng Zhou, Liang Mou, Gabriel Raphael Garcia Montoya and Istvan Podor, Pinterest Engineering, 24 Jun 2026.
19. **Timely Stream Processing**, Apache Flink documentation.
20. **Apache Spark _SUCCESS anatomy**, Bartosz Konieczny, waitingforcode.com.
21. **Interface Snapshot**, Apache Iceberg 1.9.0 Javadoc.
22. **Apache Iceberg Exactly-Once Ingestion: A Data Engineer's Guide**, RisingWave blog.
23. **Computing Extremely Accurate Quantiles Using t-Digests**, Ted Dunning and Otmar Ertl, arXiv, 11 Feb 2019.
24. **Flink Sink: Add WriteObserver plugin interface for per-record metadata (#15783)**, apache/iceberg, GitHub.
25. **Flink Sink V2: Add PostCommitHook plugin interface (#15768)**, apache/iceberg, GitHub.
26. **Redis security advisory (AV26-423), Update 1**, Canadian Centre for Cyber Security, updated 19 Oct 2026.
27. **Security advisory: CVE-2026-23479, CVE-2026-25243, CVE-2026-25588, CVE-2026-25589, CVE-2026-23631**, Redis, 5 May 2026; **CVE-2026-25243**, Red Hat; **ACL**, Redis docs.
28. **FEDORA-2026 valkey 8.1.7 security update**, LWN (upstream release 5 May 2026); **Fedora 44 valkey 9.0.4 advisory**, LinuxSecurity.
29. **kafka-storage-api 4.4.0**, Maven Repository (dated 14 Oct 2026).
30. **Upgrade to Kafka 4.4.0 (#52247)**, spring-projects/spring-boot, 15 Oct 2026.
31. **Apache Kafka downloads** and **blog**, viewed 20 Oct 2026.
32. **Snowflake Online Feature Store (GA)**, Snowflake release notes, 20 Oct 2026.
33. **DuckDB release calendar**, viewed 20 Oct 2026.
34. **Community Extensions are moving to DuckDB v2.0.0 (#3081)**, duckdb/community-extensions, 15 Oct 2026.
35. **Openflow endpoints (GA)**, Snowflake release notes, 16 Oct 2026.
36. **Databricks platform release notes, October 2026**, Databricks, entry dated 16 Oct 2026.